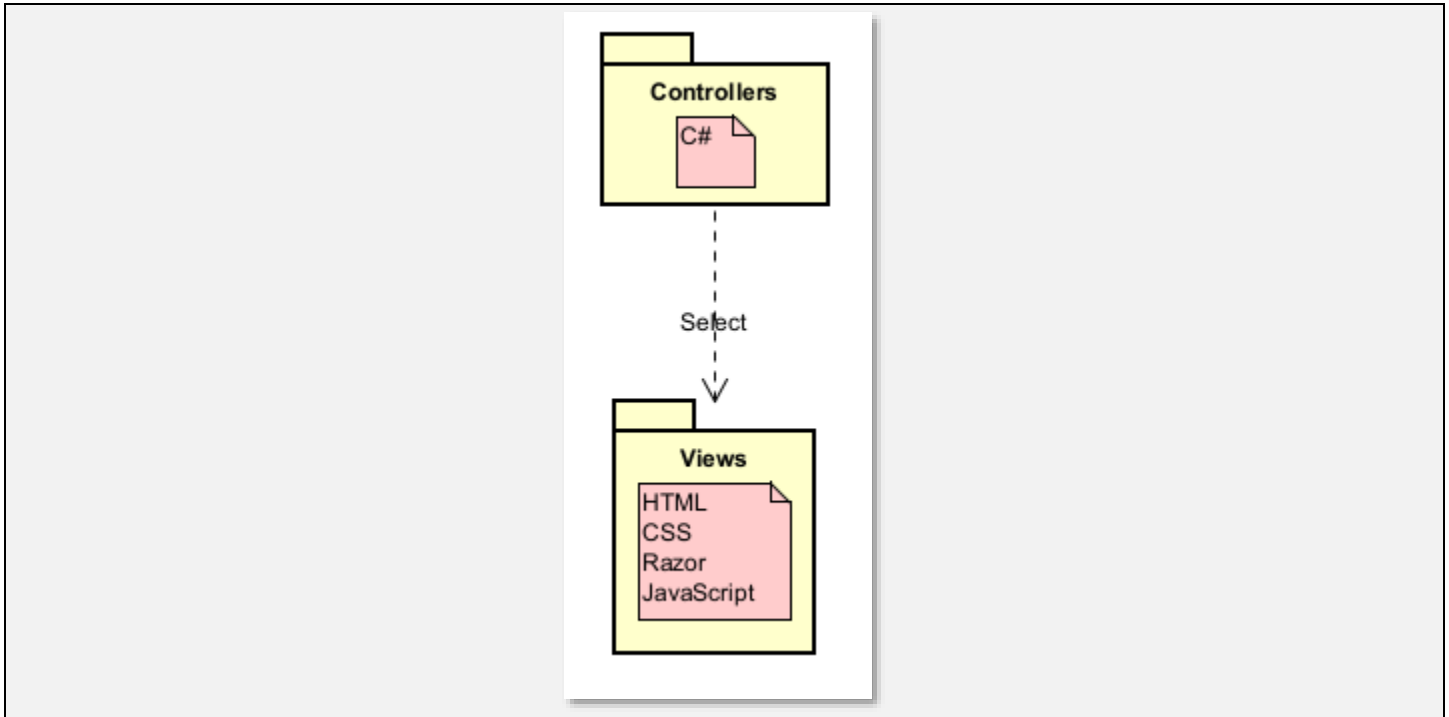


## Razor Code

- ❖ Provides a simple programming syntax for writing server-side code in Web Pages
- ❖ Uses the C# programming language and makes use of the @ symbol

## Where Razor Fits



## Server-Side Comments

- ❖ @\* ... \*@

```
<!-- This is a client-side comment -->
@* This is a server-side comment *@
Some text
@*
    This is another server-side comment
*@
```

## Code Block

- ❖ @{ ... }
- ❖ Allows you to write blocks of C# code

```
@{
    string greet = "hello";
    char gender = 'M';
    double val = 12.3;
```

```
    val = val * 100;
}
```

## Inline Expression

- ❖ @<expression>
- ❖ Evaluates expressions within the web page

```
<p>@greet @gender @val</p>
<p>@greet Jeff. You are @gender. Value is @val.</p>
```

## Code and Markup

- ❖ @<C# keyword>{ ... }
- ❖ Uses a C# control structure to create a code block

```
@if (gender == 'M')
{
    <h3>Male</h3>
}
else
{
    <h3>Other</h3>
}

@for(int i = 2; i <= 20; i+=2)
{
    <span>@i</span>
}

@{
    string[] names = {"Jeff", "Evelyn", "Darrien", "Dana", "Dylan"};
}

<ul>
@foreach (var n in names)
{
    <li>@n</li>
}
</ul>
```

- ❖ If plain text immediately follows the opening statement, it needs either <text> or @:.

```
@{
    var foo = true;
    var bar = "some value";
}

@if (foo)
{
    Plain Text ← ERROR!
}
```

```
@if (foo)
{
    <text>Plain Text</text>
}
<br />
@if (foo)
{
    @:Plain Text and @bar
}
```

## Explicit Expression

- ❖ @( <expression> )
- ❖ Evaluates the entire expression before rendering

```
345 * 67 % 34 = @(345 * 67 % 34)
@@ was escaped
```

## Layout Views

- ❖ Provide a way of specifying common view code that is visible throughout the application
- ❖ The code for the layout is placed in *\_Layout.cshtml*, which is typically placed in the *Shared* folder
- ❖ The name of the layout view is entered in the views by using the *Layout* property of the view

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";

    Layout = null; // There is no layout view
}
```

## \_ViewStart.cshtml

- ❖ To avoid specifying the layout in every view file, *\_ViewStart.cshtml* is used.
- ❖ The MVC framework uses this convention to specify the common layout code.
- ❖ This file is created in the root of the *Views* folder so that all the views will use that file.
- ❖ This file can be overridden by creating *\_ViewStart.cshtml* in the individual folders.

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
}
```

## \_ViewImports.cshtml

- ❖ The *\_ViewImports.cshtml* file is used to specify namespaces commonly used in the views.

```
@using LecRazorIntro
@using LecRazorIntro.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

## Tag Helpers

- ❖ Are ASP.NET Core extensions to the HTML tags.
- ❖ Make the view code easier to read and understand thus making it more maintainable.
- ❖ The `@addTagHelper` directive is inserted in the `_ViewImports.cshtml` file.
  - This specifies the assembly from which the tag helpers are loaded.

## ViewData and ViewBag

- ❖ Dynamic view data dictionary
- ❖ **You** create the key-value pair
  - `ViewData["programmer defined name"] = value;`
  - `ViewBag.programmer_defined_name = value;`

### In the controller

```
public IActionResult Index()
{
    ViewData["SomeValue1"] = "Used ViewData in the controller";
    ViewBag.SomeValue2 = "Used ViewBag in the controller";
    return View();
}
```

### In the view

```
<p>@ViewData["SomeValue1"]</p>
<p>@ViewBag.SomeValue1</p>
<p>@ViewData["SomeValue2"]</p>
<p>@ViewBag.SomeValue2</p>
```

## RenderBody()

- ❖ Renders the portion of a content page that is not within a named section

```
@RenderBody()
```

## RenderSectionAsync()

- ❖ Renders the content of a named section

### In the layout page:

```
@await RenderSectionAsync("SomeSectionName", required: false)
```

### In the view page:

```
@section SomeSectionName{
    Something to render
}
```

## Partial Views

(Smith, Latham, Jendoubi, Anderson, & Sauber, 2018)

- ❖ Partial view file names often begin with an underscore (\_).
  - This naming convention isn't required, but it helps to visually differentiate partial views from views and pages.

## Partial Tag Helper

- ❖ The Partial Tag Helper renders content **asynchronously** and uses an HTML-like syntax:  
`<partial name="_PartialName" />`
- ❖ When a file extension is present, the Tag Helper references a partial view that must be in the same folder as the markup file calling the partial view:  
`<partial name="_PartialName.cshtml" />`
- ❖ The following example references a partial view from the app root.
  - Paths that start with a tilde-slash (~/) or a slash (/) refer to the app root:  
`<partial name="~/Views/Folder/_PartialName.cshtml" />`  
`<partial name="/Views/Folder/_PartialName.cshtml" />`
- ❖ The following example references a partial view with a relative path:  
`<partial name="../Account/_PartialName.cshtml" />`

|                                                                               |
|-------------------------------------------------------------------------------|
| <code>_MyPartial.cshtml</code>                                                |
| <code>&lt;h1&gt;Partial View&lt;/h1&gt;</code>                                |
| <code>&lt;p&gt;This was rendered from a partial view.&lt;/p&gt;</code>        |
| <code>&lt;partial name="_MyPartial" /&gt;</code>                              |
| <div><h1>Partial View</h1><p>This was rendered from a partial view.</p></div> |

## References

- Anderson, R. (2017, April 13). *Adding validation*. Retrieved March 27, 2018, from Microsoft Docs: <https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/validation>
- Anderson, R. (2017, February 14). *View components*. (Microsoft) Retrieved October 30, 2017, from Microsoft Docs: <https://docs.microsoft.com/en-us/aspnet/core/mvc/views/view-components>
- Smith, S., Latham, L., Jendoubi, M., Anderson, R., & Sauber, S. (2018, September 10). *Partial views in ASP.NET Core*. Retrieved October 15, 2018, from Microsoft Docs: <https://docs.microsoft.com/en-us/aspnet/core/mvc/views/partial?view=aspnetcore-2.1>